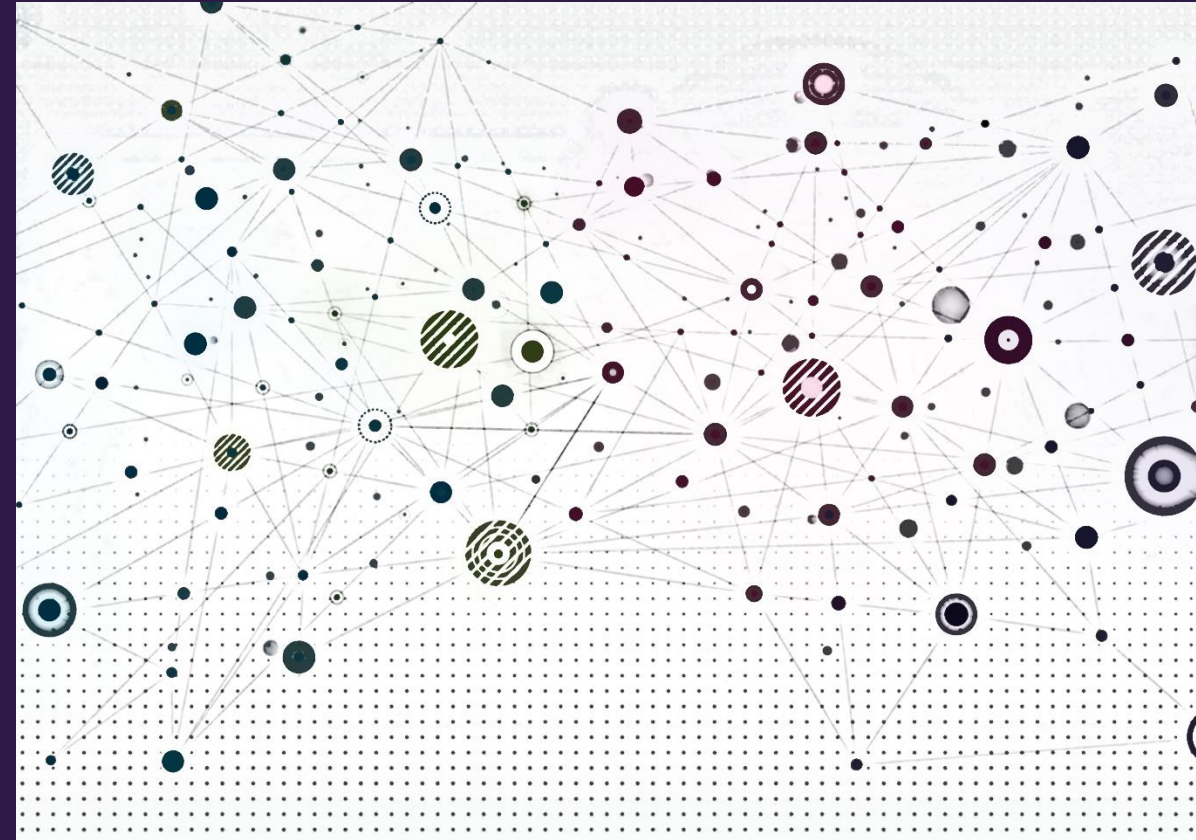


GNNs in practice





Overview

- Recap
- GNN prediction heads
- GNN training
- GNN performance evaluation
- Splitting our dataset



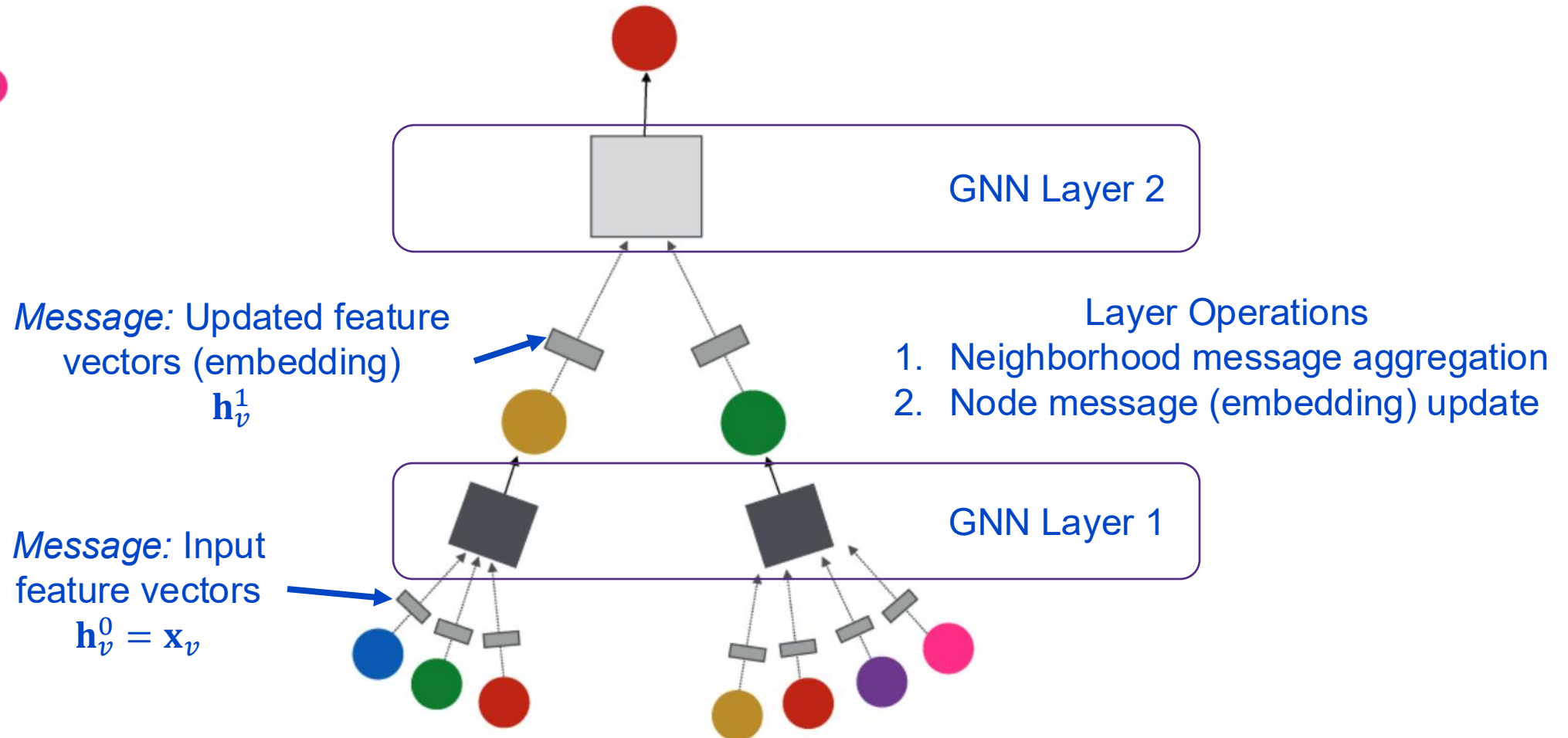
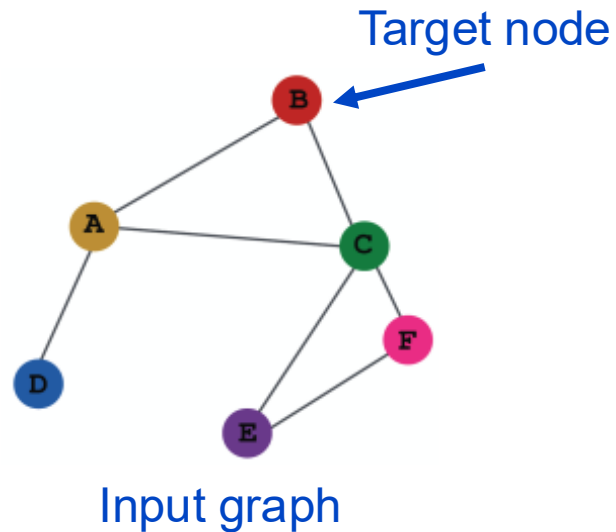
Recap



Message passing in GNNs

GNN Layer $\equiv$ Aggregation $\rightarrow$ Update

- Different architectures under this perspective
- GCN, GraphSage, GAT, ...





Message aggregation and update as a GNN layer

- Combine the neighborhood message aggregation and node embedding update to form a single GNN layer

1. **Aggregation**: each node aggregates messages (embeddings) of neighbor nodes

$$\mathbf{m}_{N(v)}^{(k)} = \text{Agg}^{(k)} \left(\left\{ \mathbf{h}_u^{(k)}, \forall u \in N(v) \right\} \right)$$

2. **Update**: each node updates its embedding (message for next layer)

$$\mathbf{h}_v^{(k+1)} = \text{Update}^{(k)} \left(\mathbf{h}_v^{(k)}, \mathbf{m}_{N(v)}^{(k)} \right)$$

- The update usually includes a **nonlinearity (activation)**, $\sigma(\cdot)$, such as ReLU, sigmoid



Graph convolutional networks

- A GCN can be written as

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\mathbf{W}^{(k)} \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|} \right)$$

Recall: Graph augmented with self loops

- How to write this as aggregation $\rightarrow$ update?

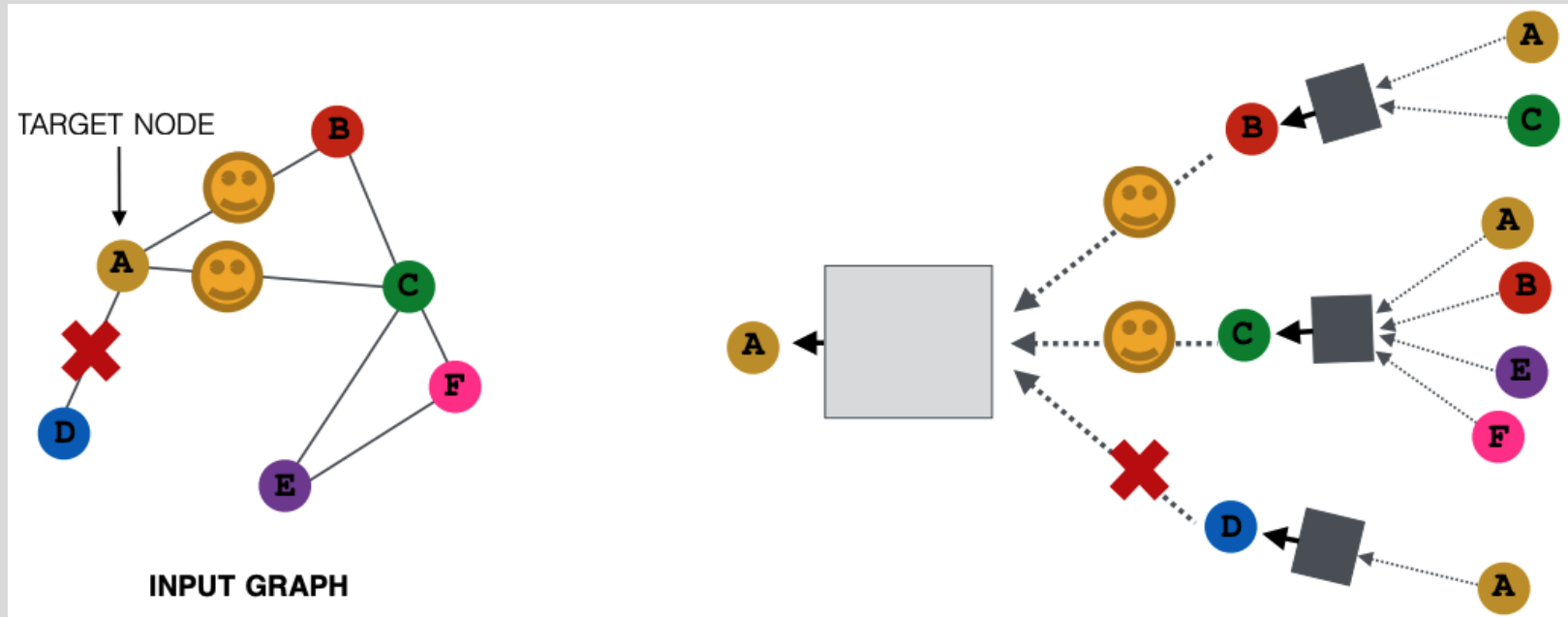
$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v) \cup \{v\}} \frac{\mathbf{h}_u^{(k)}}{|N(v)|}$$

$$\mathbf{h}_v^{(k+1)} = \sigma \left(\mathbf{W}^{(k)} \mathbf{m}_{N(v)}^{(k)} \right)$$



GraphSAGE (SAmple and aggreGatE) sampling

- In expectation (on average), we get embeddings similar to aggregating over all neighbors
 - Greatly reduces computational cost



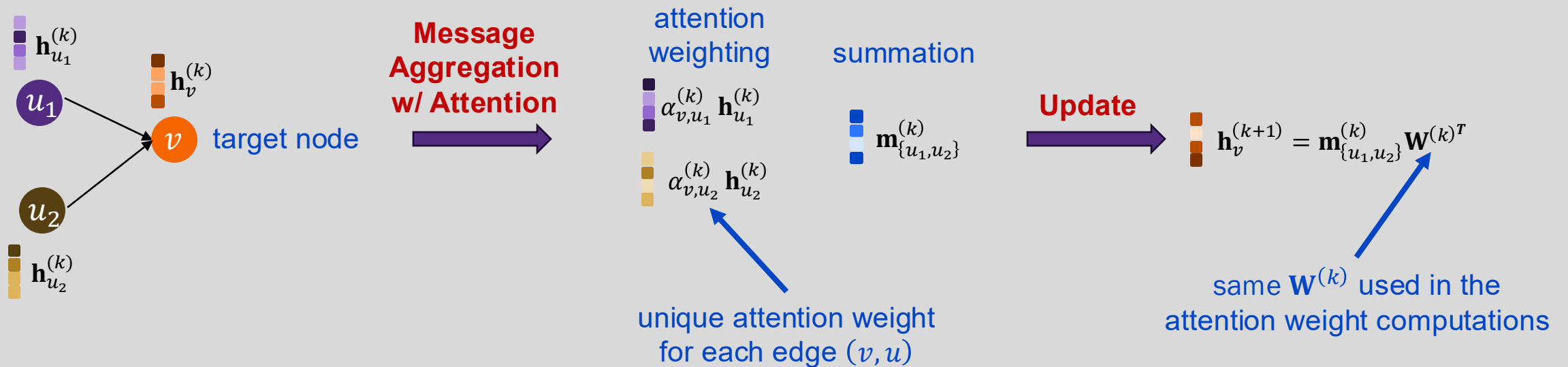


Graph attention with convolution update

- Formulate attention in the message passing framework

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k)} \mathbf{h}_u^{(k)}$$

$$\mathbf{h}^{(k+1)} = \mathbf{m}_{N(v)}^{(k)} \mathbf{W}^{(k)T}$$



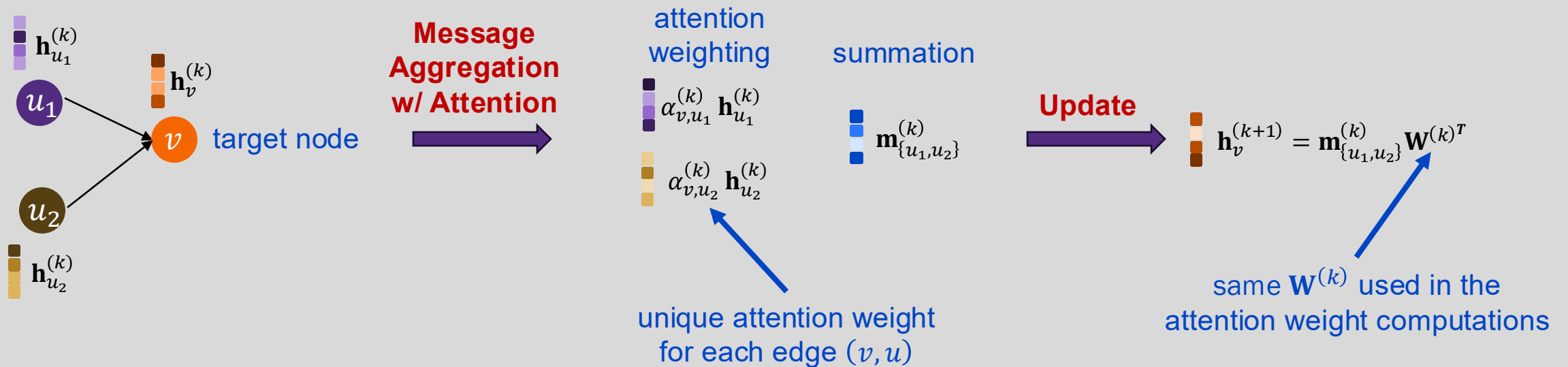


Graph attention with convolution update

- Formulate attention in the message passing framework

$$\mathbf{m}_{N(v)}^{(k)} = \sum_{u \in N(v)} \alpha_{v,u}^{(k)} \mathbf{h}_u^{(k)}$$

$$\mathbf{h}^{(k+1)} = \mathbf{m}_{N(v)}^{(k)} \mathbf{W}^{(k)T}$$





Why manipulate graphs

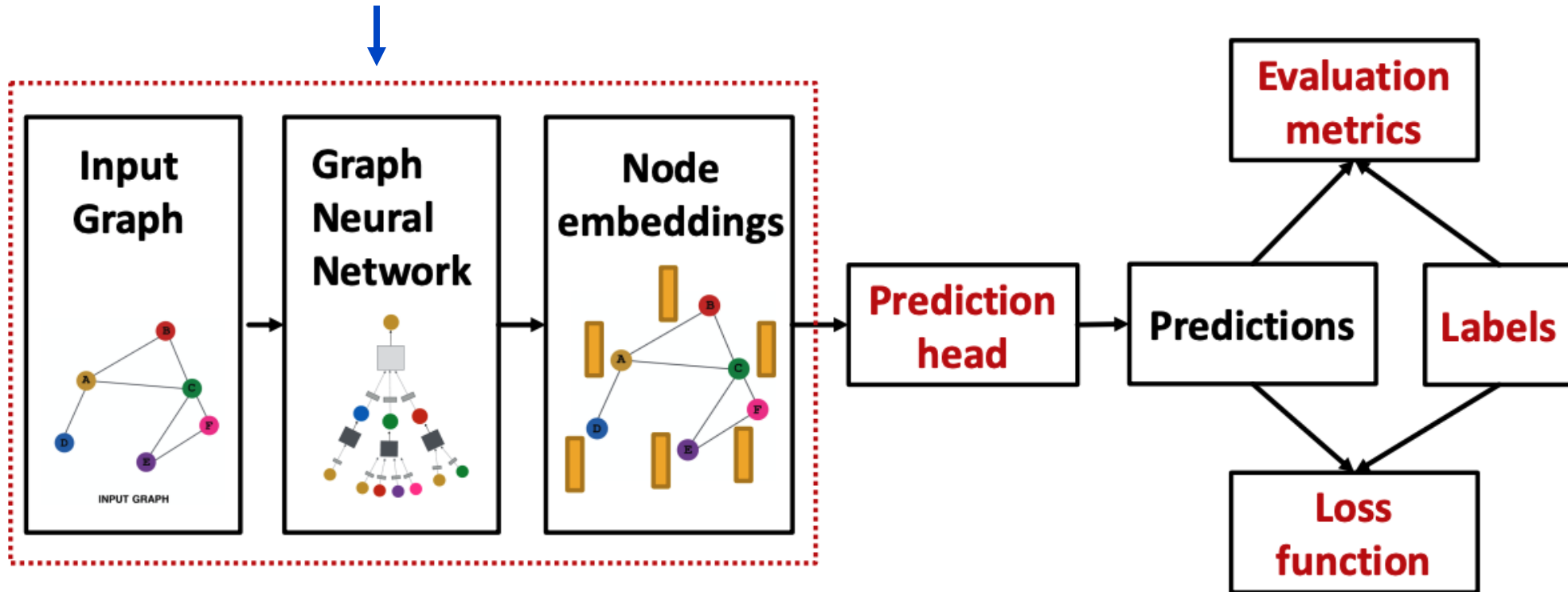
- We've assumed that the network graph = the GNN structure
- We might want to break this assumption if:
 - The input graph lacks features → apply feature augmentation (later lecture)
 - The graph is too sparse → add virtual nodes
 - The graph is too dense → apply GraphSAGE or other neighbor sampling
 - The graph is too large → sample subgraphs to compute embeddings (later lecture, time permitting)



GNN prediction heads



We've learned about these

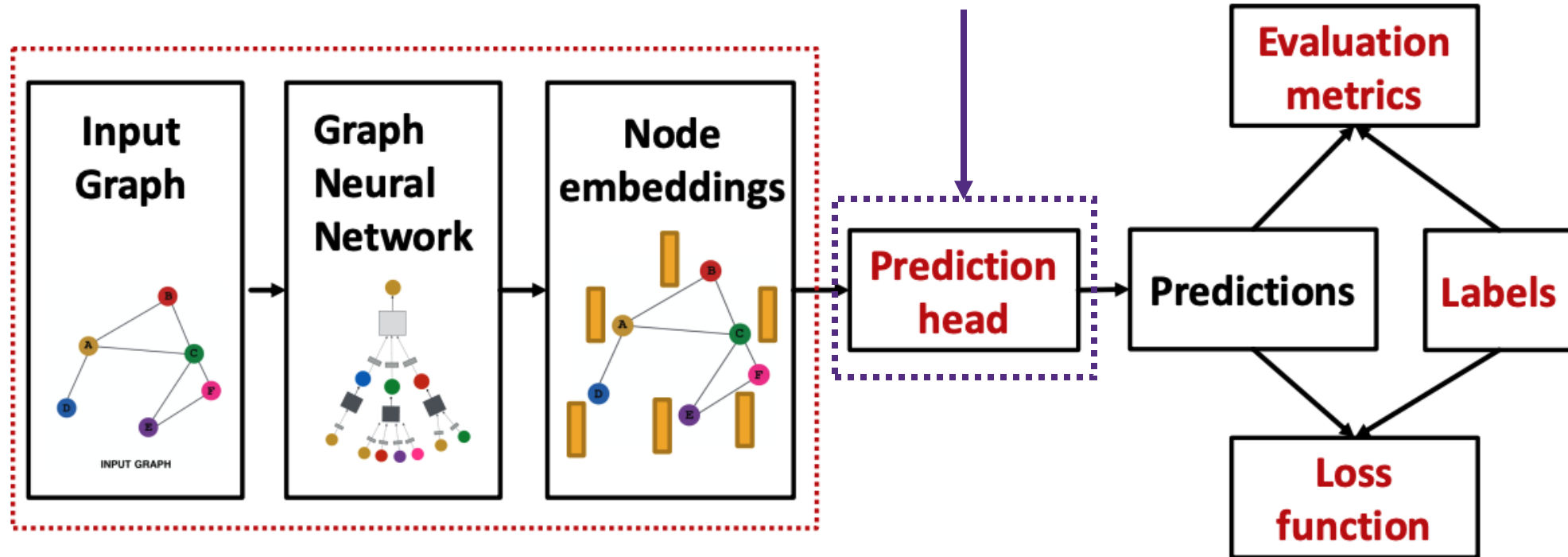


GNN output: a set of node embeddings
 $\{\mathbf{h}_v^{(K)} \mid \forall v \in V\}$

What do we do with these?

Add “prediction heads”
(additional network layers) for:

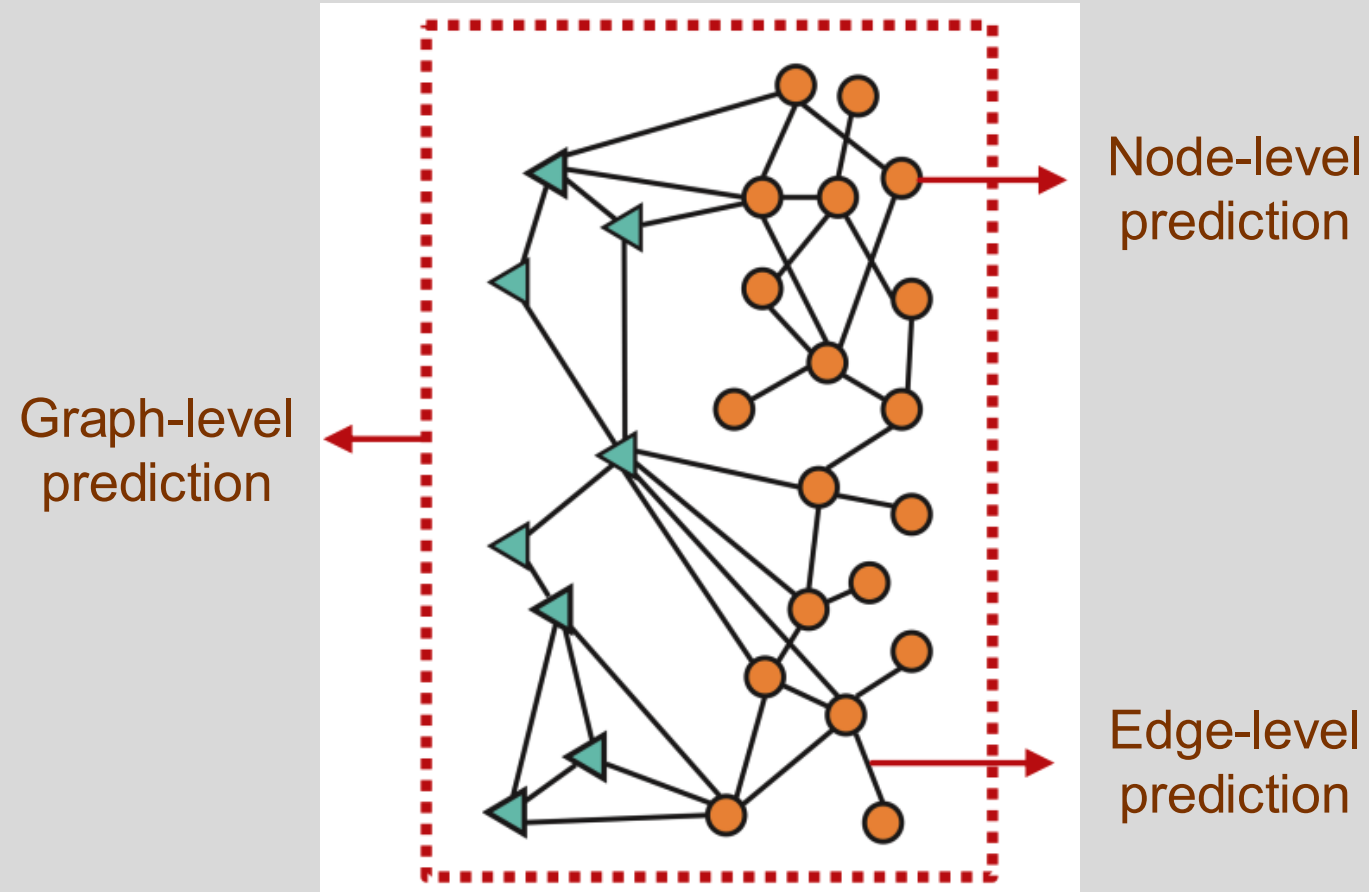
- node-level tasks
- edge-level tasks
- graph-level tasks





GNN prediction heads

Different tasks levels require different types of prediction heads





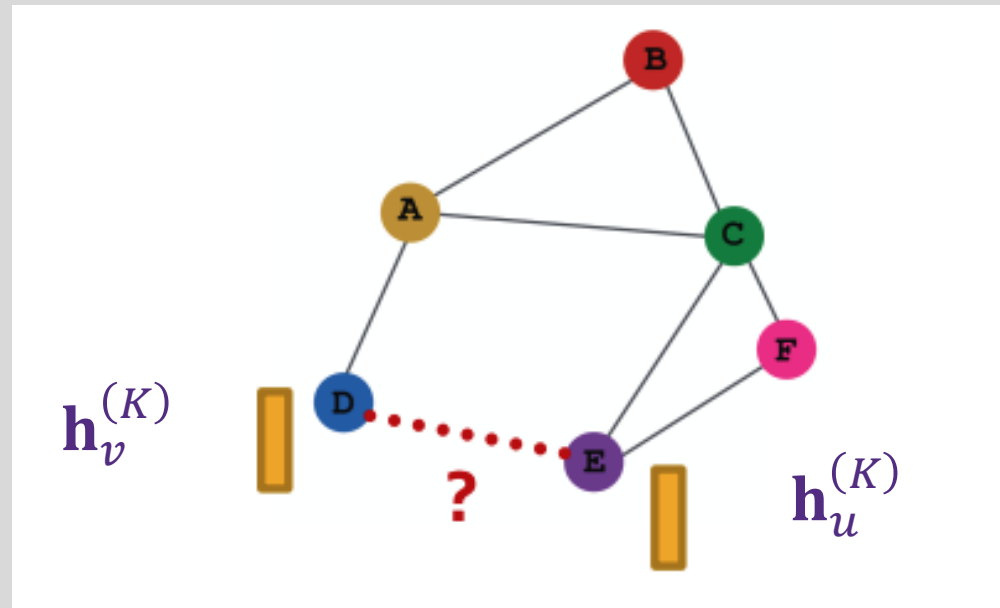
Node-level prediction head

- Use the node embedding as input to a prediction function (e.g., an MLP)
- Following the GNN computation with K layers, we have a d_K -dimensional embeddings: $\{\mathbf{h}_v^{(K)} \in \mathbb{R}^{d_K}, \forall v \in V\}$
- Assume we want to make a **C -way prediction**
 - Classification: classify among C categories
 - Regression: regress on C variables
- Create a prediction head, g , that outputs and estimate, $\hat{\mathbf{y}}_v$, of the target, from $\mathbf{h}_v^{(K)}$
 - Example: g is a linear layer, $\hat{\mathbf{y}}_v = g(\mathbf{h}_v^{(K)}) = W^g \mathbf{h}_v^{(K)}$
 - $W^g \in \mathbb{R}^{C \times d_K} \rightarrow$ maps node embeddings from $\mathbb{R}^{d_K}$ to $\mathbb{R}^C$
 - We can now compute a loss function on the node prediction task!



Edge-level prediction head

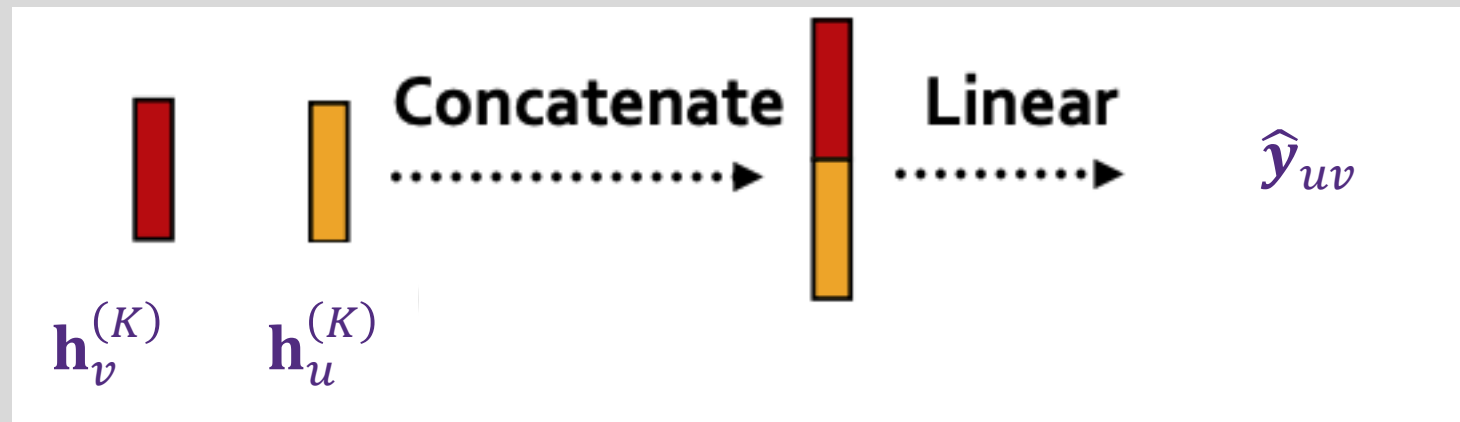
- Use **pairs** of node embeddings as input to a prediction function
- Assume we want to make a **C -way prediction**
- $\hat{y}_{uv} = g(\mathbf{h}_v^{(K)}, \mathbf{h}_u^{(K)})$
- What are the options for $g(\mathbf{h}_v^{(K)}, \mathbf{h}_u^{(K)})$?





Edge-level prediction head with concatenation

- One option is to concatenate the u and v node embeddings
- Prediction function, g , operates on the concatenated vector
- Example: g is a linear layer, $\hat{y}_{uv} = g \left(\left[\mathbf{h}_u^{(K)} \mid \mathbf{h}_v^{(K)} \right] \right) = W^g \left[\mathbf{h}_u^{(K)} \mid \mathbf{h}_v^{(K)} \right]$



- Here, $W^g \in \mathbb{R}^{C \times 2d_K}$ → maps node embeddings from $\mathbb{R}^{2d_K}$ to $\mathbb{R}^C$

Account for the concatenation
of embeddings



Edge-level prediction head with dot product

- The prediction head function is just the dot product of the node embeddings
 - $\hat{y}_{uv} = \mathbf{h}_u^{(K)T} \cdot \mathbf{h}_v^{(K)}$
 - This **only applies to 1-way prediction** (i.e., binary classification, single regression, link prediction)
- Applying to **C-way** prediction with a **learnable generalized dot product**
 - Learn c weight matrices

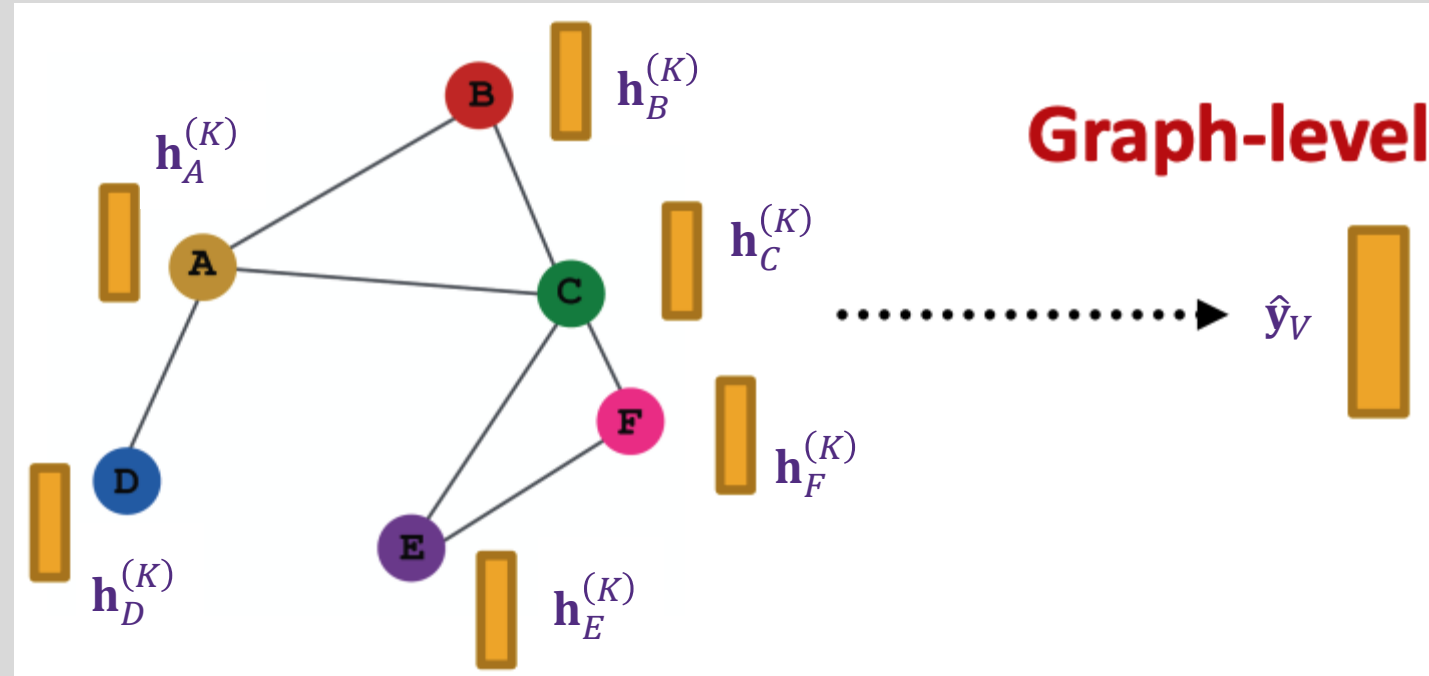
$$\begin{aligned}\hat{y}_{uv,1} &= \mathbf{h}_u^{(K)T} \mathbf{W}^1 \mathbf{h}_v^{(K)} \\ &\vdots \\ \hat{y}_{uv,c} &= \mathbf{h}_u^{(K)T} \mathbf{W}^c \mathbf{h}_v^{(K)} \\ \hat{\mathbf{y}}_{uv} &= [\hat{y}_{uv,1}, \dots, \hat{y}_{uv,c}] \in \mathbb{R}^c\end{aligned}$$

where $\mathbf{W}^c \in \mathbb{R}^{d_K \times d_K}$



Graph-level prediction head

- Assume we want to make a C -way prediction
- $\hat{\mathbf{y}}_G = g \left(\left\{ \mathbf{h}_v^{(K)} \in \mathbb{R}^{d_K}, \forall v \in V \right\} \right)$
- Options for g are similar to message aggregation and update

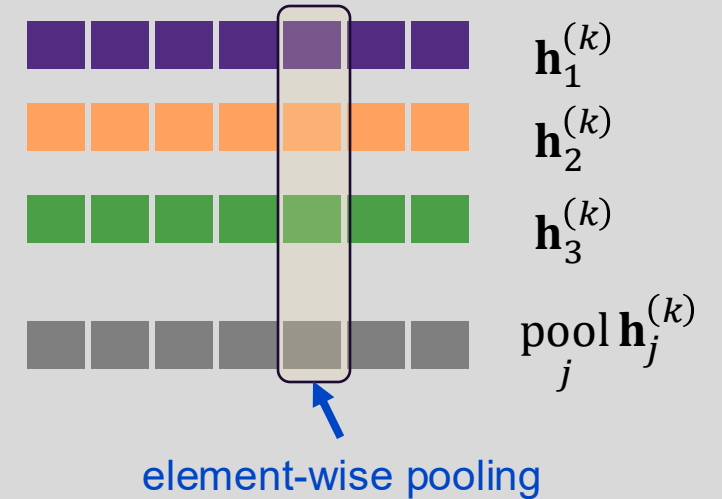




Graph-level prediction head via pooling

- **Element-wise** pooling (aggregation) over node embeddings

- Mean pooling: $\mathbf{h}_G^{(K)} = \text{Mean} \left(\left\{ \mathbf{h}_v^{(K)} \in \mathbb{R}^{d_K}, \forall v \in V \right\} \right)$
- Max pooling: $\mathbf{h}_G^{(K)} = \text{Max} \left(\left\{ \mathbf{h}_v^{(K)} \in \mathbb{R}^{d_K}, \forall v \in V \right\} \right)$
- Sum pooling: $\mathbf{h}_G^{(K)} = \text{Sum} \left(\left\{ \mathbf{h}_v^{(K)} \in \mathbb{R}^{d_K}, \forall v \in V \right\} \right)$



- Prediction head operates on pooled embedding

$$\hat{\mathbf{y}}_G = g \left(\mathbf{h}_G^{(K)} \right)$$

- Example: g is a linear layer, $\hat{\mathbf{y}}_G = g \left(\mathbf{h}_G^{(K)} \right) = W^g \mathbf{h}_G^{(K)}$
 - $W^g \in \mathbb{R}^{C \times d_K} \rightarrow$ maps node embeddings from $\mathbb{R}^{d_K}$ to $\mathbb{R}^C$
- Works well for small graphs. What about large graphs?



Limitations for global pooling

- Global pooling loses information
- Toy example with 1-dimensional node embeddings for two graphs each with 5 nodes
 - Node embeddings for G_1 : $\{-1, -2, 0, 1, 2\}$
 - Node embeddings for G_2 : $\{-10, -20, 0, 10, 20\}$
 - G_1 and G_2 have different embeddings $\rightarrow$ we should be able to differentiate them with appropriate prediction head
- But, if we do global sum pooling:
 - $\hat{\mathbf{y}}_{G_1} = g\left(\mathbf{h}_{G_1}^{(K)}\right) = g(\text{sum}\{-1, -2, 0, 1, 2\}) = g(0)$
 - $\hat{\mathbf{y}}_{G_2} = g\left(\mathbf{h}_{G_2}^{(K)}\right) = g(\text{sum}\{-10, -20, 0, 10, 20\}) = g(0)$
 - We cannot differentiate G_1 and G_2



Hierarchical global pooling

- Aggregate the node embeddings hierarchically
- Toy example from previous slide with hierarchical aggregation with ReLU
 - Separately aggregate first 2 nodes and last 3 nodes
 - Aggregate again for final prediction
- Node embeddings for $G_1: \{-1, -2, 0, 1, 2\}$
 - Round 1: $h_{G_1,a} = \text{ReLU}(\text{sum}(-1, -2)) = 0$, $h_{G_1,b} = \text{ReLU}(\text{sum}(0, 1, 2)) = 3$
 - Round 2: $h_{G_1} = \text{ReLU}(\text{sum}(h_{G_1,a}, h_{G_1,b})) = 3$
- Node embeddings for $G_2: \{-10, -20, 0, 10, 20\}$
 - Round 1: $h_{G_2,a} = \text{ReLU}(\text{sum}(-10, -20)) = 0$, $h_{G_2,b} = \text{ReLU}(\text{sum}(0, 10, 20)) = 30$
 - Round 2: $h_{G_2} = \text{ReLU}(\text{sum}(h_{G_2,a}, h_{G_2,b})) = 30$
- $\hat{\mathbf{y}}_{G_1} = g(3)$, $\hat{\mathbf{y}}_{G_2} = g(30)$. We can differentiate these!



Differentiable pooling

- Idea: learn a hierarchical node pooling

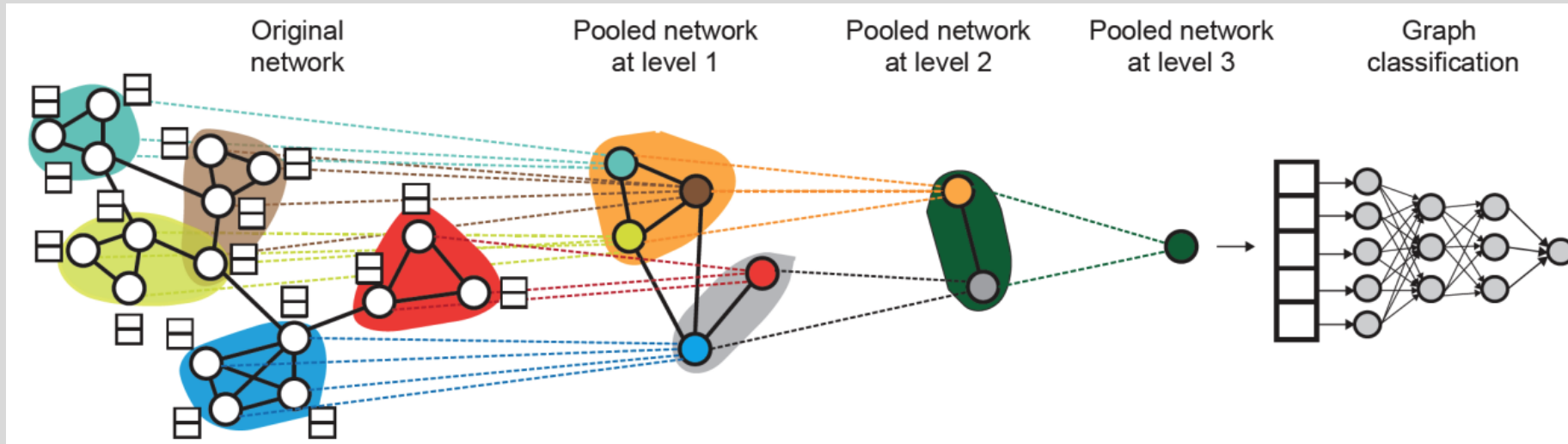


Image credit: Ying et. al. 2019

- Leverage 2 independent GNNs at each pooling level
 - GNN 1 computes node embeddings
 - GNN 2 computes node cluster membership
- GNN 1 and 2 at each level can be executed in parallel



Differentiable pooling

- Idea: learn a hierarchical node pooling

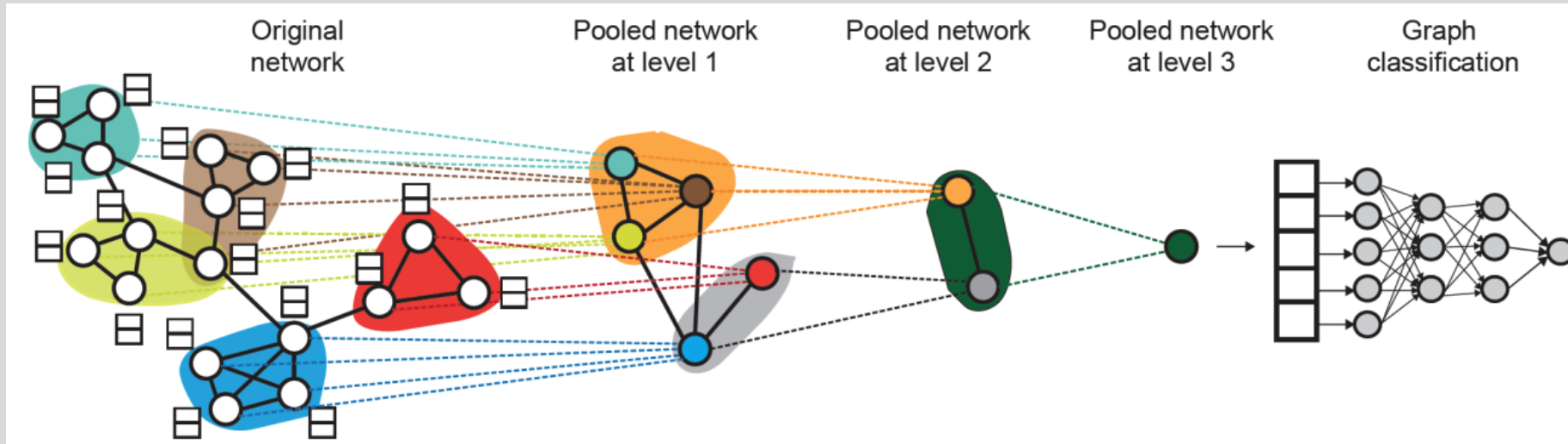


Image credit: Ying et. al. 2019

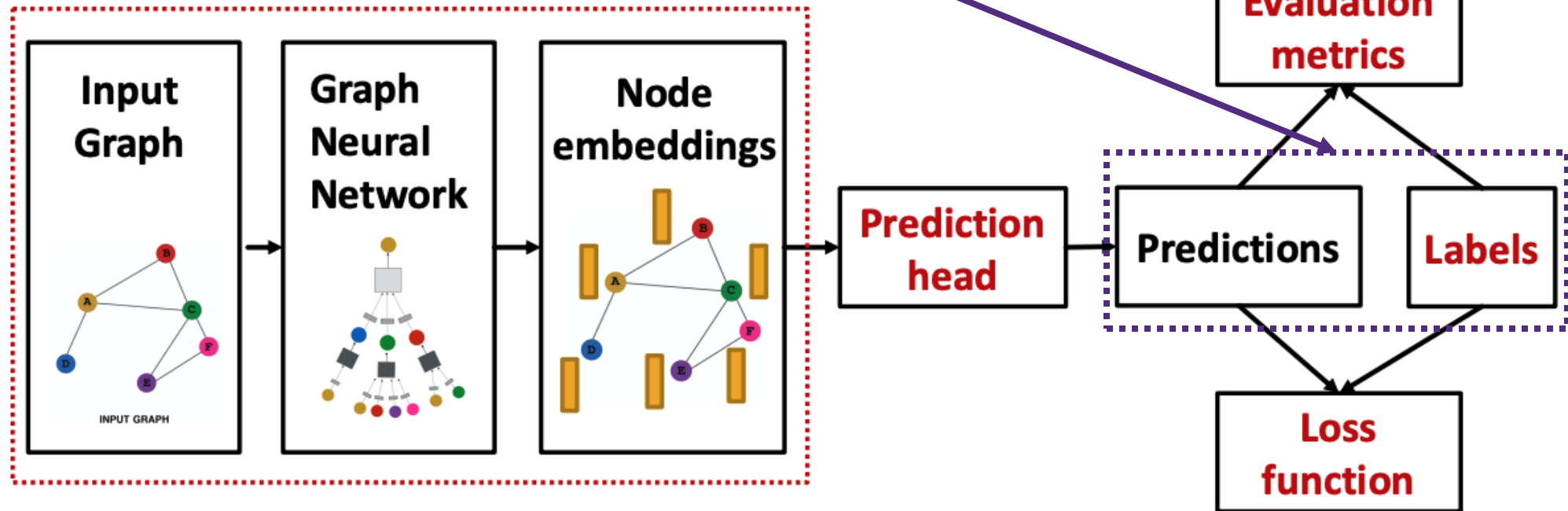
- For each pooling layer
 - Use cluster assignments to aggregate node embeddings
 - Create a single new node for each cluster, maintaining edges between clusters to generate next level pooled network
- Jointly train GNN 1 and GNN 2



GNN training



Where do ground-truth labels come from?





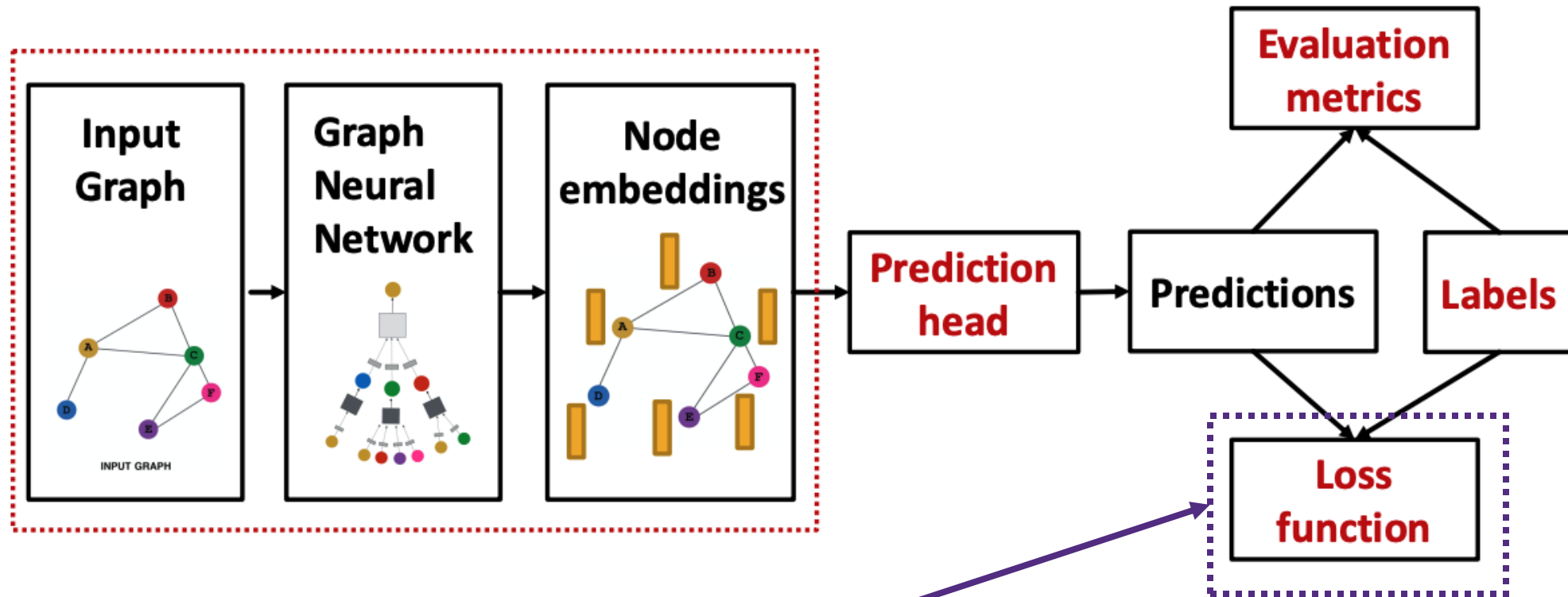
Supervised learning

- Supervised learning – labels come from an external source
- Examples
 - Node labels: article topic in a citation network
 - Edge labels: fraudulent transaction in a financial transaction network
 - Graph labels: toxicity of a drug



Unsupervised learning

- Unsupervised learning – labels come from the graph structure
- Examples
 - Node labels: node statistics such as a centrality measure
 - Edge labels: link prediction based on input graph structure
 - Graph labels: graph statistics such as diameter, isomorphism (pair of graphs)
- Technically, we have supervision in unsupervised
- Unsupervised is more appropriately called self-supervised



How do we compute the final loss?



GNN training setting

- We have N data points
 - Each data point, i , can be a node/edge/graph
 - Node-level: prediction $\hat{\mathbf{y}}_v^{(i)}$, label $\mathbf{y}_v^{(i)}$
 - Edge-level: prediction $\hat{\mathbf{y}}_{uv}^{(i)}$, label $\mathbf{y}_{uv}^{(i)}$
 - Graph-level: prediction $\hat{\mathbf{y}}_G^{(i)}$, label $\mathbf{y}_G^{(i)}$
- To simplify notation, we'll drop the subscripts and use predictions $\hat{\mathbf{y}}^{(i)}$ and label $\mathbf{y}^{(i)}$ for all task data types



Classification or regression

- Classification: labels $\mathbf{y}^{(i)}$ are categorical values (no objective ordering)
- Regression: labels $\mathbf{y}^{(i)}$ are numerical (typically continuous, ordered)
- GNNs are applicable in both settings but with different loss functions and evaluation metrics



Classification loss functions

- Cross entropy (CE) is the most common
- Binary CE (two classes) for the i^{th} data point

$$BCE(y^{(i)}, \hat{y}^{(i)}) = y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})$$

- C -way CE (C classes) for the i^{th} data point

$$CE(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) = - \sum_{j=1}^C \mathbf{y}_j^{(i)} \log \hat{\mathbf{y}}_j^{(i)} = -\mathbf{y}^{(i)} \cdot \hat{\mathbf{y}}^{(i)}$$

where $\mathbf{y}^{(i)} \in \mathbb{R}^C$ is a one-hot label encoding, $\hat{\mathbf{y}}^{(i)} \in \mathbb{R}^C$ is prediction after Softmax

- Total loss over all N training samples

$$\mathcal{L} = \sum_{i=1}^N BCE(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$

- There are many other loss functions – we'll cover them as needed



Regression loss function

- Most often use Mean Squared Error (MSE)
- C -way MSE (C outcome variables) for the i^{th} data point

$$MSE(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) = - \sum_{j=1}^C \left(\mathbf{y}_j^{(i)} - \hat{\mathbf{y}}_j^{(i)} \right)^2$$

where $\mathbf{y}^{(i)} \in \mathbb{R}^C$ is a real valued vector of target outcomes,
 $\hat{\mathbf{y}}^{(i)} \in \mathbb{R}^C$ is a real valued vector of predicted outcomes

- Total loss over all N training samples

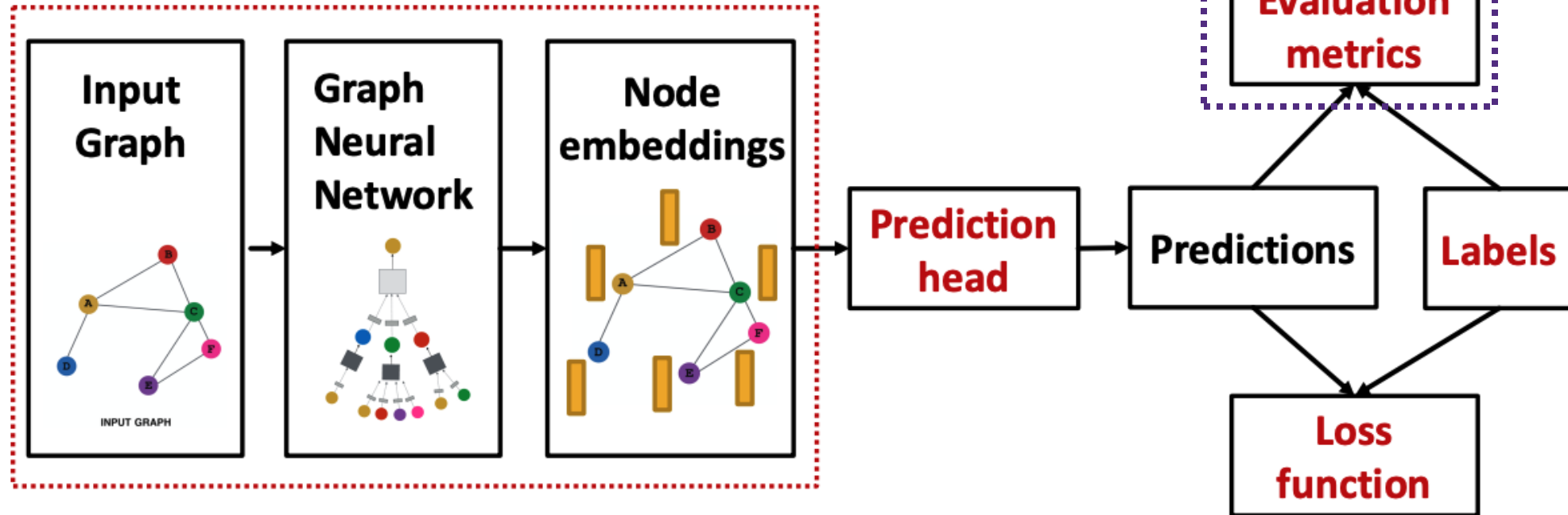
$$\mathcal{L} = \sum_{i=1}^N MSE(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$



GNN performance evaluation



How do we measure
performance?





Evaluation metrics for regression

- Use standard machine learning evaluation metrics
- Assume we make predictions for M data points
- Root mean square error (RMSE)

$$\sqrt{\frac{1}{M} \sum_{i=1}^M (\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)})^2}$$

- Mean absolute error

$$\frac{1}{N} |\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)}|$$

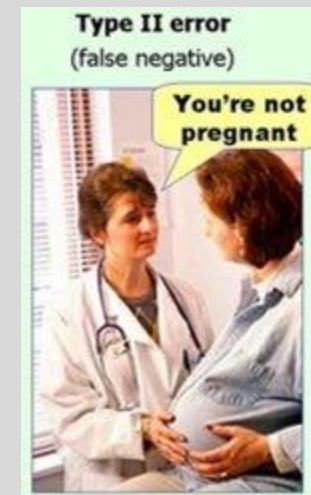
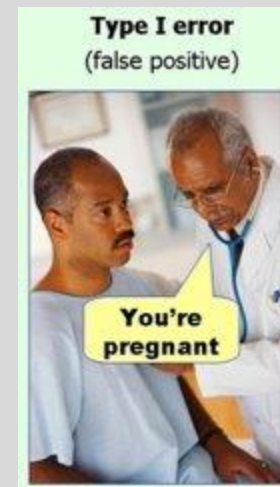
- Note, for C -way regression, these result in a vector – one entry for each target outcome



Classifier error types

All classifier performance metrics are derived from counts of TP, FP, TN, FN

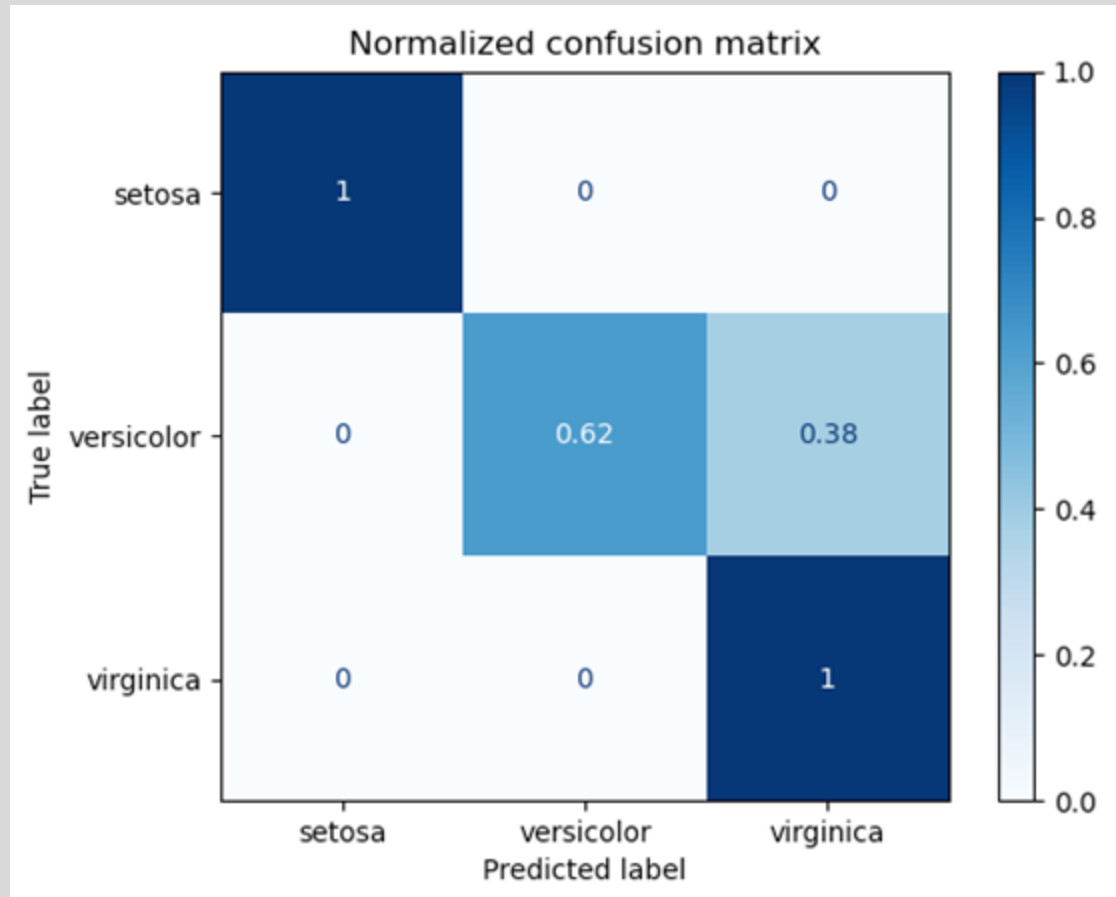
		Actual Class	
		Positive	Negative
Predicted Class	Positive	True Positive (TP)	False Positive (FP) <i>False Alarm</i> <i>Type I Error</i>
	Negative	False Negative (FN) <i>Type II Error</i>	True Negative (TN)





Confusion Matrix

We can identify how correctly classified were the observations by computing a Confusion Matrix that shows the correct and misclassified observations.





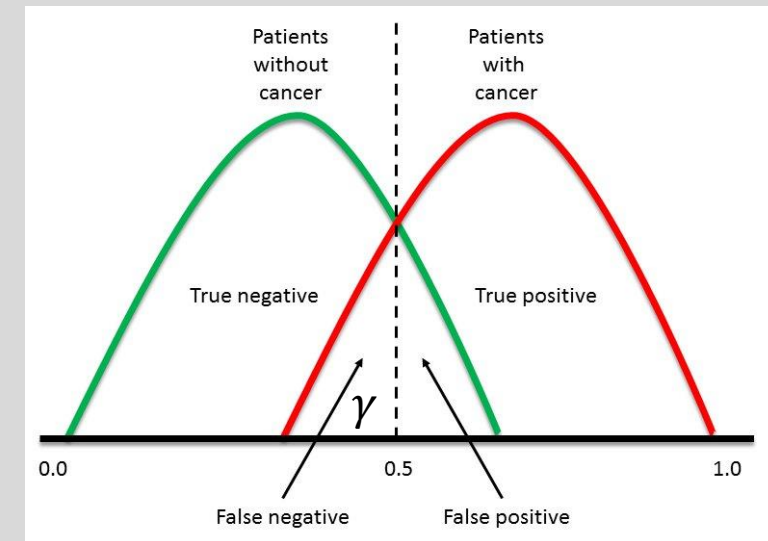
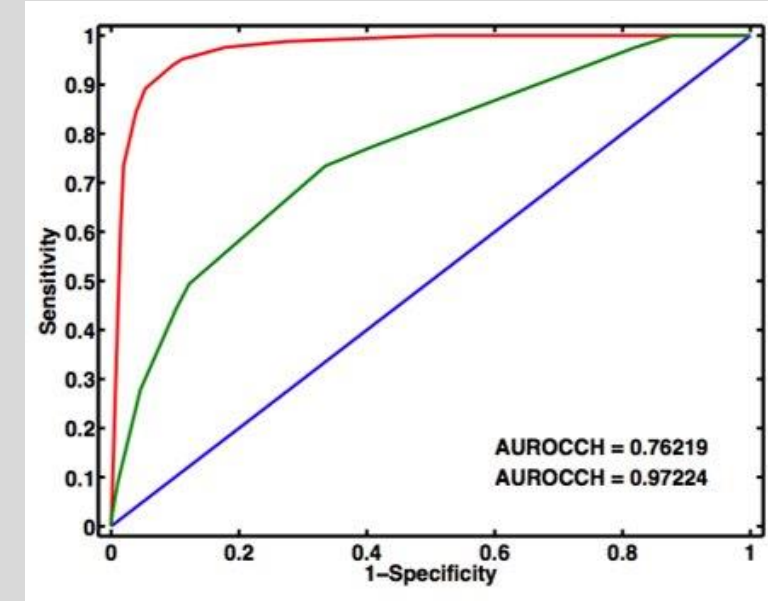
Classifier performance point metrics

- Accuracy = $(TP+TN)/(TP+TN+FP+FN)$
- Sensitivity (a.k.a Recall, True Positive Rate) = $TP/(TP+FN) = TP/P$
 - The fraction of actually positive samples predicted positive by the model
- Specificity (a.k.a. True Negative Rate) = $TN/(TN+FP) = TN/N$
 - The fraction of actually negative samples predicted negative by the model
- Precision (a.k.a. Positive Predictive Value) = $TP/(TP+FP)$
 - The fraction of samples predicted positive by the model that are actually positive
- False Positive Rate (a.k.a. Probability of false alarm) = $FP/(FP+TN) = FP/N$
- F1 Score = $2(Precision * Recall)/(Precision + Recall)$
- Balanced Accuracy = $(Sensitivity + Specificity)/2$



Classifier performance with ROC and AUC

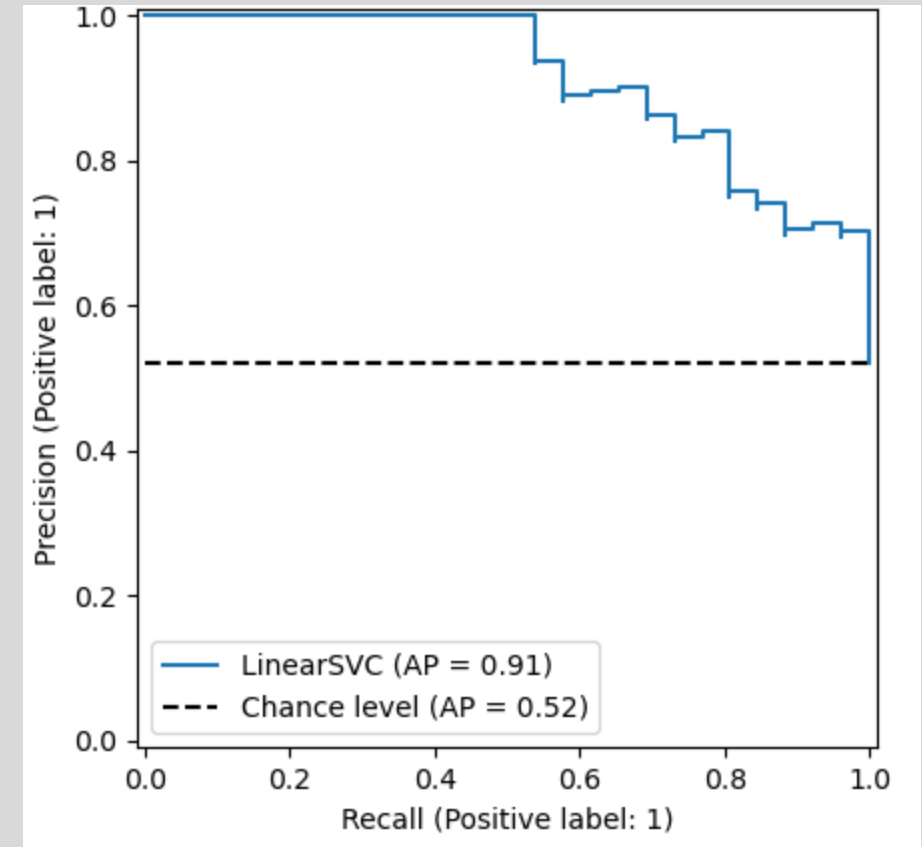
- Recall, we can select $0 < \gamma < 1$ as a threshold s.t. $\hat{P}(Y = 1) \geq \gamma$ implies $\hat{y} = 1$. What happens if we change γ ?
- Receiver Operating Characteristic Curve (ROC)
 - Illustrates tradeoff between FPR and TPR of a binary classifier as the discrimination threshold, γ , is adjusted
 - Axes
 - $1 - \text{Specificity}$ (false positive rate) = $1 - \text{TN}/N$
 - Sensitivity (true positive rate) = TP/P
- Area under the curve (AUC)
 - Summary statistic
 - AUC = 0.5 indicates no predictive value (random guessing)
 - AUC = 1.0 indicates perfect predictive value





Classifier performance with precision-recall curve

- Illustrates tradeoff between precision (PPV) and sensitivity of a binary classifier as the discrimination threshold, γ , is adjusted
- Particularly useful for highly imbalanced datasets (i.e., where one class is very rare). In those cases, it's possible to achieve high AUC with very low precision*
- Axes
 - Sensitivity (true positive rate) = TP/P
 - Precision (PPV) = $TP/(TP+FP)$
- Average precision – summary metric indicating the weighted average precision for each discrimination threshold



*Romero-Brufau S, Huddleston JM, Escobar GJ, Liebow M. Why the C-statistic is not informative to evaluate early warning scores and what metrics to use. Crit Care. 2015 Aug 13;19(1):285



Splitting our dataset





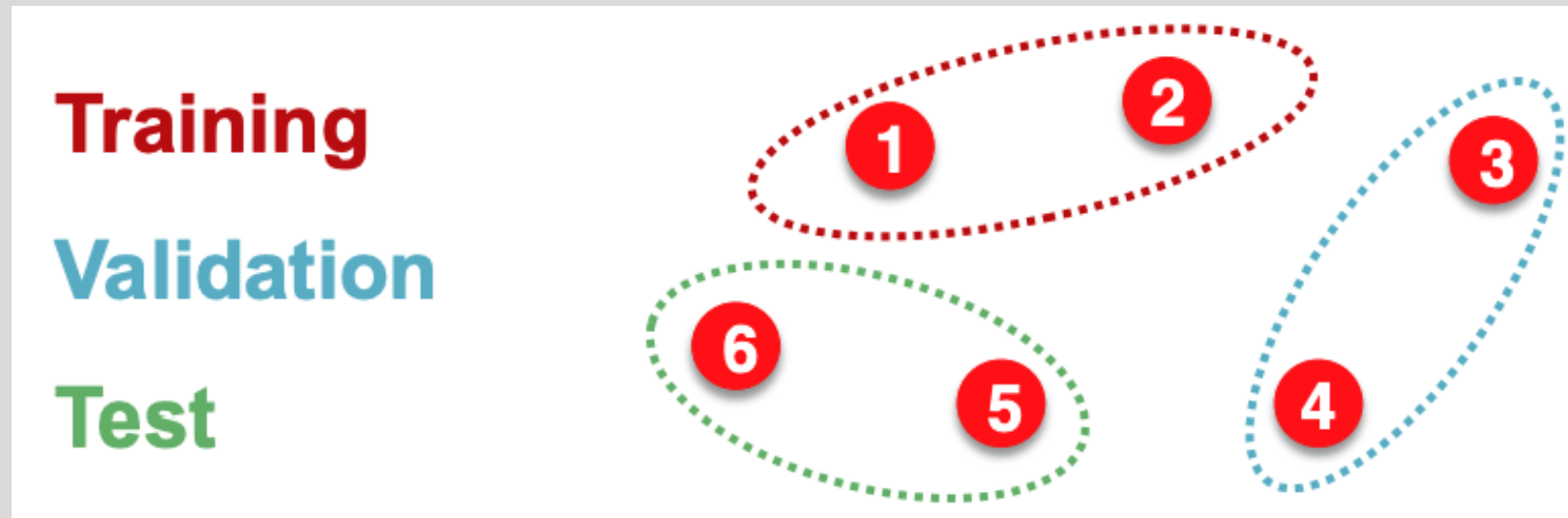
Dataset splits

- Assume we're working with a large dataset and not using cross-validation
- Randomly split the data into:
 - Training set: used for optimizing GNN and prediction head parameters
 - Validation set: used to develop model (different architectures) and tune hyperparameters
 - Test set: held out until model(s) are finalized to report final performance
- If feasible, we should repeat the process and report **average performance over many different random splits**
- **Concern:** Can we guarantee that the test set is really held out?



Graph data splitting is special

- Suppose we are splitting an image dataset for image classification
- Each data point is an image
- Data points are independent
 - Image 5 will not affect our prediction for image 1





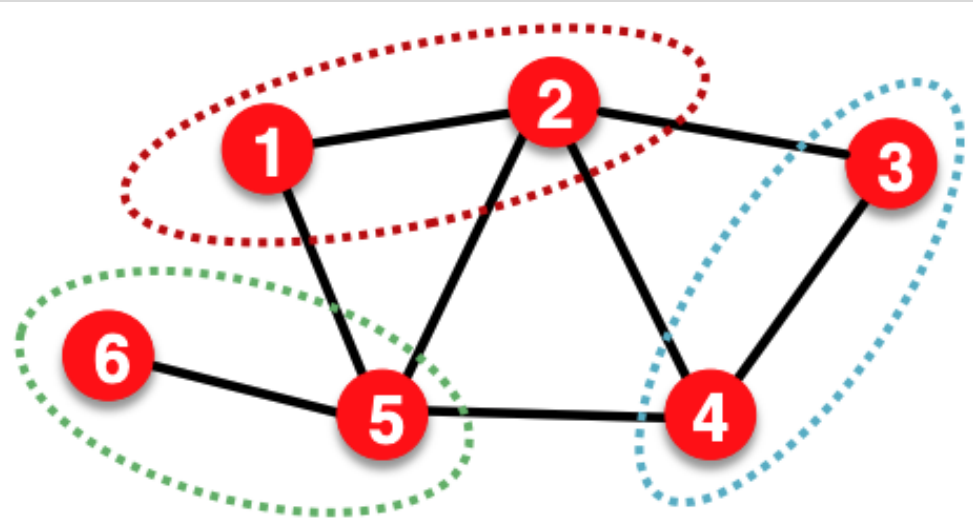
Graph data splitting is special

- Assume we wish to perform node classification
- Each data point is a node
- Data points are **NOT independent**
 - Node 5 **will** affect our prediction for node 1 because it will participate in message passing that will affect node 1's embedding

Training

Validation

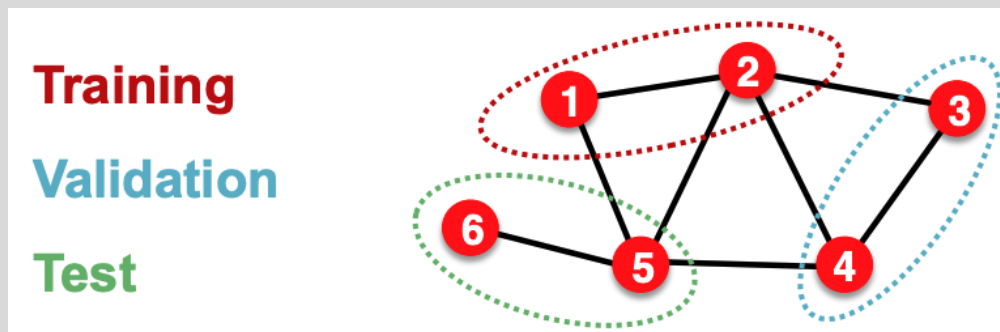
Test





Transductive setting data splits

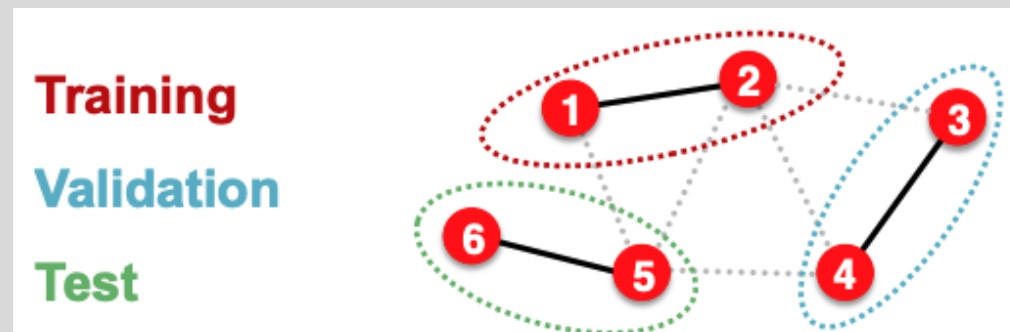
- The input graph is observed in all data splits (train, validation, test)
- Only the node labels are split
 - **Training time**, compute embeddings using the entire graph via message passing, evaluate the prediction head performance and update the model parameters using training nodes (1 & 2 in example)
 - **Validation time**, compute embeddings using the entire graph and evaluate the prediction head performance on validation nodes (3 & 4 in example)





Inductive setting data splits

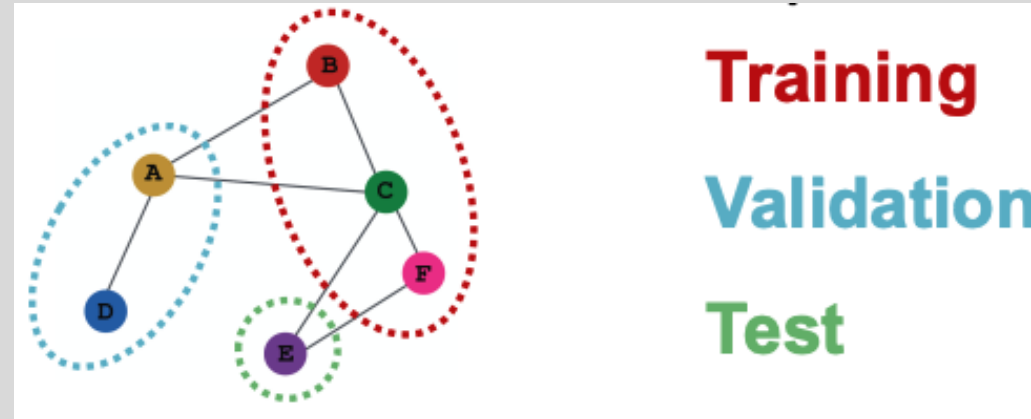
- Break edges between the splits (generates multiple graphs / components)
 - Now we have three independent graphs
 - **Training time**, compute embeddings using only the training graph, evaluate the prediction head performance and update model parameters using training nodes (1 & 2 in example)
 - **Validation time**, compute embeddings for the validation nodes and evaluate performance on validation nodes (3 & 4 in example)



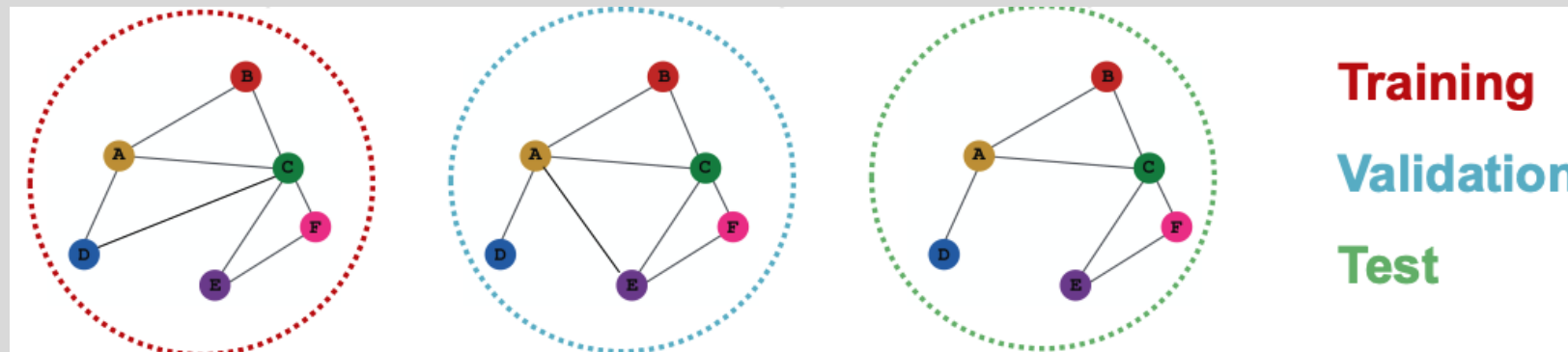


Node classification

- **Transductive** – all splits can observe the entire graph structure, but can only observe the labels of their respective nodes



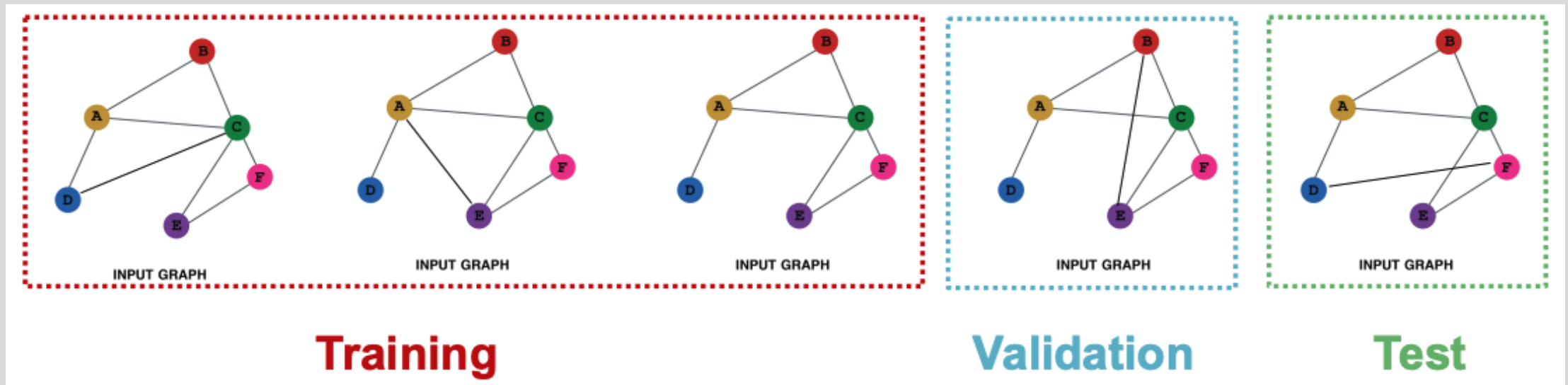
- **Inductive** – each split contains an independent graph





Graph classification

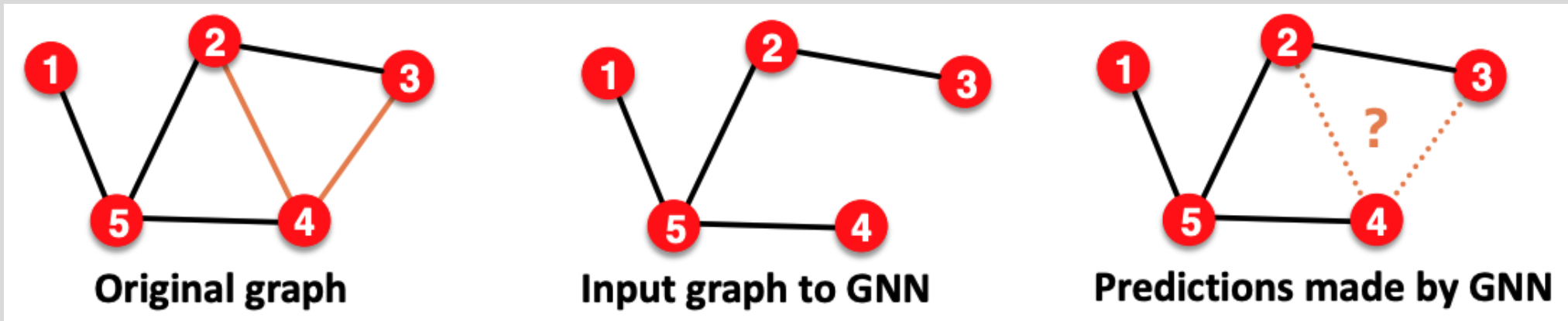
- Only the **inductive** setting is applicable because we **must** test on unseen graphs





Edge (link) prediction

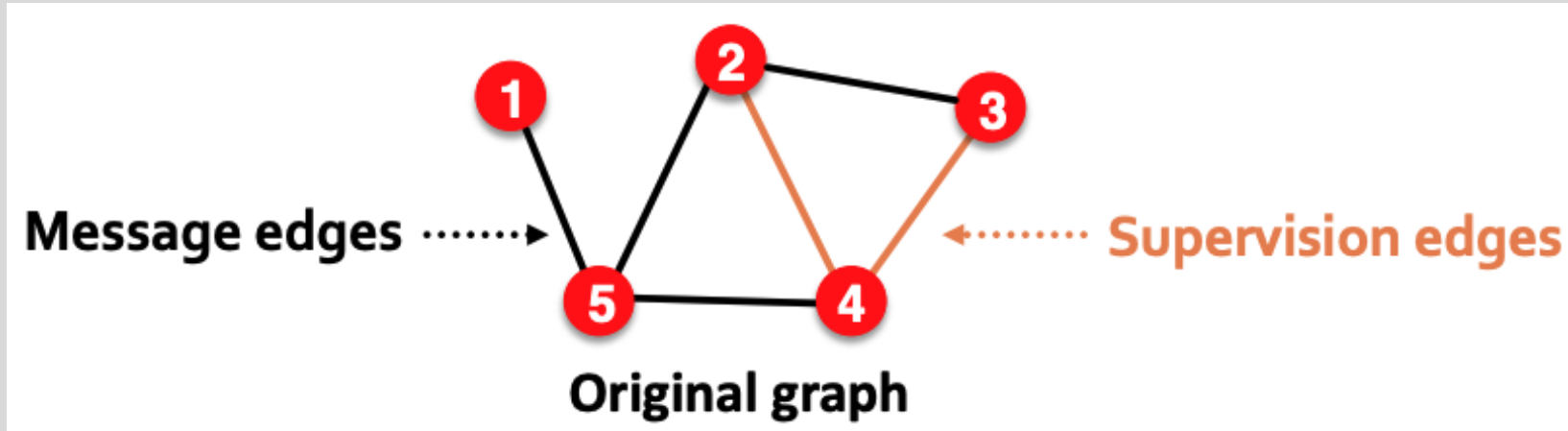
- Goal is to predict missing edges
- We need to **hide some edges** in the input graph from the GNN and **let the GNN + prediction head determine if the edge should exist**





Setting up edge prediction

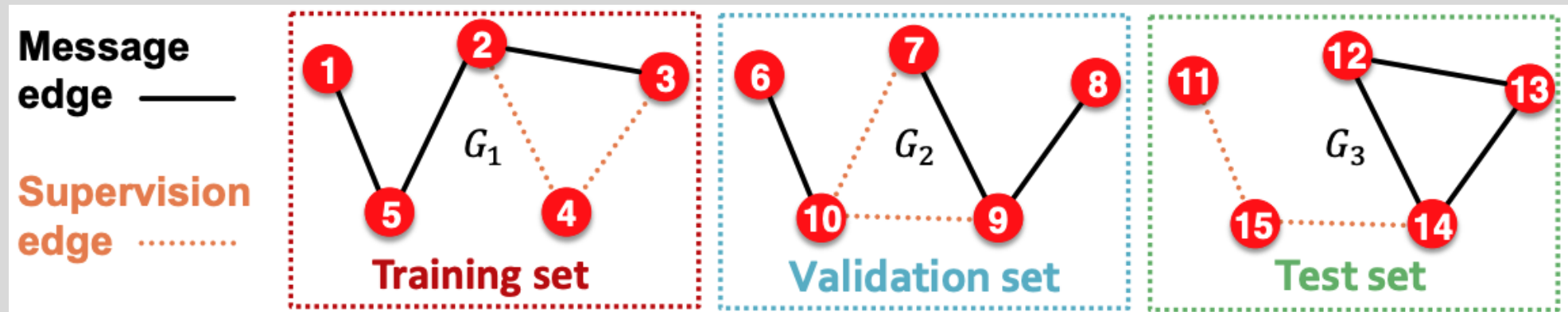
- We will need to split the edges twice
- **First split:** assign 2 types of edges in the input graph
 - Message edges: will be used for GNN message passing
 - Supervision edges: will be used for training and evaluation
- After the first split:
 - Only message edges will remain in the input graph
 - Supervision edges are not fed into the GNN





Setting up edge prediction

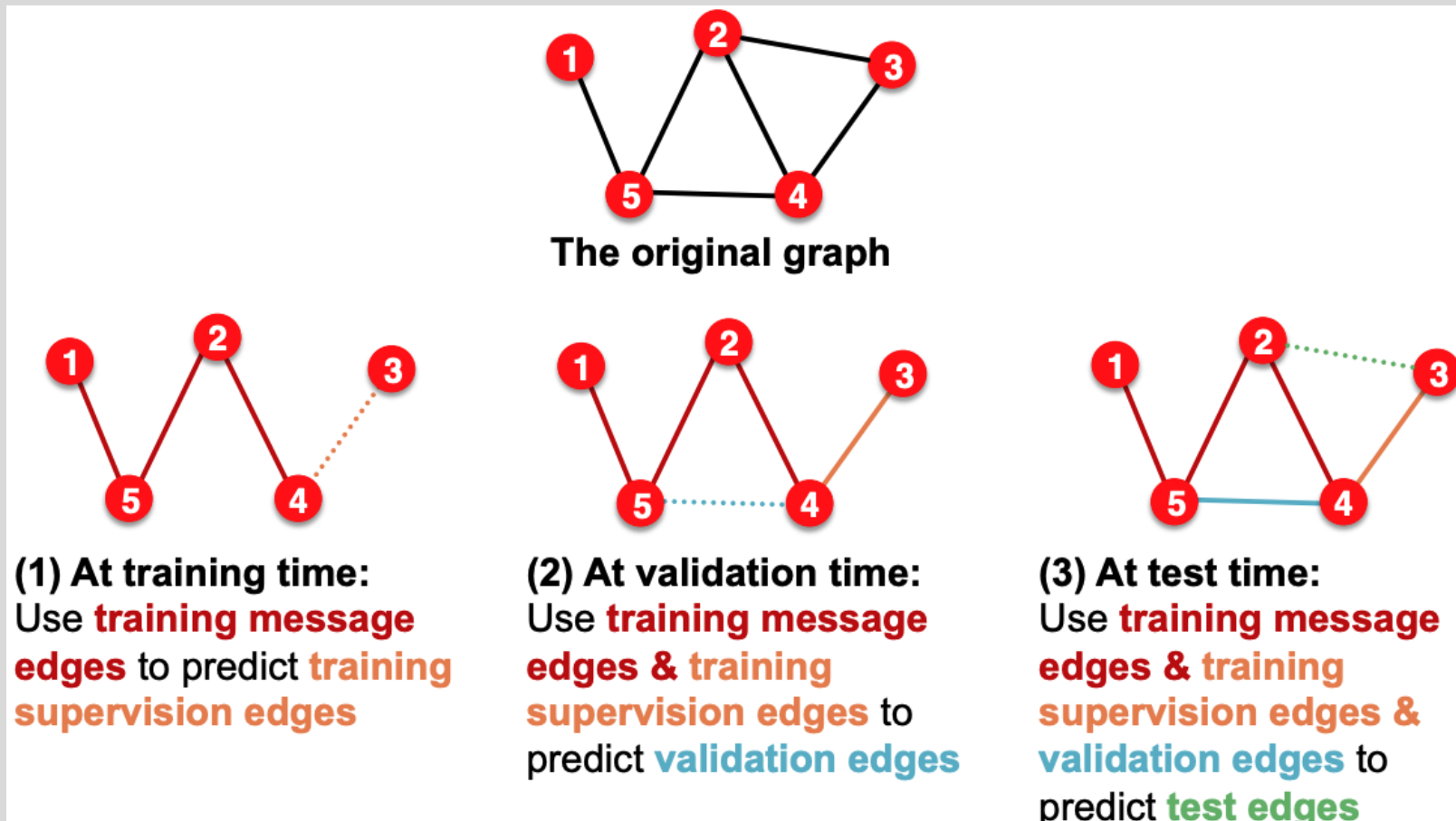
- **Option 1: Inductive second split:** assign each message & supervision edge to one of train / validation / test
- The assigned edges will form three independent graphs
- Each graph will have message and supervision edges





Setting up edge prediction

- **Option 2: Transductive second split** (most common choice)
 - All message edges used in training, supervision edges split among train / val / test





Setting up edge prediction

- **Option 2:** Transductive second split (most common choice)
- Link prediction settings are tricky. You may see variations in papers.

